

Chapter 3: Frontend Data & State Management

Welcome back to the CinéConnect tutorial! In [Chapter 2: User Authentication](#), we learned how users securely log in, register, and maintain their sessions, building a personalized and protected experience. Now that we know who our users are, it's time to tackle how we efficiently manage and display all the dynamic information they see, like movie lists, user profiles, and search results.

Smart Data Handling: Why It Matters on the Frontend

Imagine you're browsing CinéConnect. You see a list of trending movies. You click on a movie to see its details, then go back to the trending list. Should the app fetch that *entire* list of movies again every single time you navigate back? What if the internet is slow, or you're on a mobile device? Constantly re-fetching data can make an app feel sluggish and waste your valuable data.

This is where "Frontend Data & State Management" comes in. It's about having smart strategies to:

- **Fetch data:** Get information from the backend (our server) or external services.
- **Cache data:** Remember data we've already fetched so we don't have to ask for it again immediately.
- **Keep data fresh:** Automatically update cached data in the background so you always see the latest information.
- **Handle loading and errors:** Show a loading spinner when waiting for data, and a clear error message if something goes wrong.
- **Manage global app state:** Keep essential, non-sensitive information (like whether a user is logged in) easily accessible across different parts of the application.

Our main goal in this chapter is to understand how CinéConnect uses specialized tools to fetch, manage, and keep application data up-to-date efficiently, making your browsing experience smooth and fast.

Let's explore the two main tools that make this possible: [React Query](#) and [Zustand](#).

The Two Master Chefs: [React Query](#) and [Zustand](#)

Think of your frontend application as a busy kitchen. You need different specialists to handle different tasks:

1. [React Query](#) - The Data Fetching & Caching Expert:

- This is like a super-efficient head chef who remembers every order you've ever placed. When you ask for the "trending movies" again, the chef first checks if they already have a fresh batch ready. If so, they serve it instantly! If not, or if the batch is getting old, they'll quickly prepare a new one, perhaps even doing it in the background while you enjoy what's already on your plate.
- [React Query](#) handles the complex parts of fetching data from our backend, showing loading states, dealing with errors, and automatically keeping that data fresh.

2. [Zustand \(useAuthStore\)](#) - The Global Recipe Book:

- This is like a small, lightweight recipe book that holds essential information everyone in the kitchen needs to know, such as "Is the customer currently logged in?" or "What's their email

address?". It's not for big, complex orders (that's **React Query**'s job), but for quick, universally needed details.

- Specifically, **useAuthStore** uses **Zustand** to manage the *public* details of the logged-in user (like their **userId** and **email**), making it instantly available anywhere in the app without re-fetching. Remember from **Chapter 2: User Authentication** that the actual secure tokens (**JWT**, **Refresh Token**) are in **httpOnly cookies**, inaccessible to JavaScript – **useAuthStore** only holds the non-sensitive public user info.

How to Get and Manage Data with **React Query**

Let's look at a concrete example: displaying a list of trending movies on the home page.

The User Experience

1. You open CinéConnect and land on the Home page.
2. You see a list of popular, trending movies.
3. (Later) You navigate to another page, then come back to the Home page.
4. The movie list appears instantly, and maybe updates subtly in the background if newer data is available.

Using **useMovieList** (powered by **React Query**)

In CinéConnect, we create custom "hooks" (special functions in React) that wrap **React Query** to make it even easier to use. For trending movies, we have **useMovieList**.

```
// apps/frontend/src/pages/Home.tsx (Conceptual)
import { useMovieList } from "@/hooks/useMovies"; // Our hook for movie lists
import { Spinner } from "@/components/ui/spinner"; // For loading indication

function HomePage() {
  const { data, isLoading, isError, error, refetch } = useMovieList();

  if (isLoading) {
    return <Spinner size="lg" />; // Show a spinner while fetching
  }

  if (isError) {
    return <p>Error loading movies: {error?.message}</p>; // Show an error message
  }

  return (
    <div>
      <h1>Trending Movies</h1>
      <button onClick={() => refetch()}>Refresh Movies</button>
      <ul>
        {data.map((movie) => (
          <li key={movie.id}>{movie.title}</li>
        ))}
      </ul>
    </div>
  );
}
```

In this snippet:

- We call `useMovieList()`. This hook immediately starts fetching the trending movies.
- `isLoading`: This variable is `true` while the data is being fetched, and `false` once it's ready (or an error occurs). We use it to show a `Spinner`.
- `isError`: This variable is `true` if an error happened during fetching, and `false` otherwise. We use it to display an error message.
- `error`: If `isError` is `true`, this will contain the error details.
- `data`: This is where our list of trending movies will be once `isLoading` is `false` and `isError` is `false`. It's an array of movie objects.
- `refetch`: This is a function you can call to force `React Query` to re-fetch the data, even if it's currently cached.

This simple hook provides all the essential states you need for a robust user experience, without you having to write any complex logic for fetching, caching, or error handling!

Example Input and Output

When you first load the `HomePage`:

- **Input:** `useMovieList()` is called.
- **Output (initial):** `isLoading` is `true`, `isError` is `false`, `data` is `[]` (or `undefined`), `error` is `null`. The user sees a `Spinner`.
- **Output (after fetch):** `isLoading` is `false`, `isError` is `false`, `data` is `[{ id: 1, title: "Movie 1" }, { id: 2, title: "Movie 2" }]`, `error` is `null`. The user sees the movie list.

If you navigate away and come back:

- `React Query` might instantly give you the `data` from its cache (`isLoading` is `false`), making the page feel super fast.
- Then, in the background, `React Query` might silently `refetch` the data to ensure it's up-to-date, updating the `data` variable if new movies are trending.

How to Manage Global User State with `Zustand` (`useAuthStore`)

For essential user details that don't change often but are needed everywhere (like their login status or email), `Zustand` offers a simple "global memory" solution. Remember from [Chapter 2: User Authentication](#) how `useAuth` was used to get `isAuthenticated`? This is where `Zustand` helps.

Using `useAuth` (powered by `useAuthStore`)

The `useAuth` hook simplifies authentication logic by leveraging `Zustand`'s `useAuthStore` to keep track of the logged-in user's *public* information.

```
// apps/frontend/src/hooks/useAuth.ts (Simplified)
import { useAuthStore, type AuthUser } from "@stores/auth.store";

export function useAuth() {
```

```

    const user = useAuthStore((s) => s.user); // Get the user info from the global
store
    const setUser = useAuthStore((s) => s.setUser); // Function to set user info
    const clearAuth = useAuthStore((s) => s.clearAuth); // Function to clear user
info

    // ... (login, logout, register logic, calling authService) ...

    const login = async (credentials: any) => {
      // After successful backend login...
      const res = { userId: 1, email: "user@example.com" }; // Mock response
      setUser({ userId: res.userId, email: res.email }); // Store public user info
globally
    };

    const logout = async () => {
      // After successful backend logout...
      clearAuth(); // Clear user info from global store
    };

    return {
      user,
      isAuthenticated: !!user, // Simple check if 'user' object exists
      login,
      logout,
      // ...
    };
  }
}

```

In this simplified `useAuth` hook:

- `useAuthStore((s) => s.user)` directly retrieves the `user` object (or `null`) from our global `Zustand` store.
- `setUser` and `clearAuth` are actions provided by `useAuthStore` to update or remove the user information in this global memory.
- The `isAuthenticated` derived from `!!user` tells us if a user object is present, meaning someone is logged in.

Any component in our app can now use `useAuth` to instantly know the login status or access the user's email, without needing to pass this information down through many components or re-fetch it.

Under the Hood: How Data and State are Managed

Let's peek behind the curtain to see how `React Query` and `Zustand` accomplish their magic.

`React Query`: The Smart Cache Manager

When you call a `useQuery` hook (like the one inside `useMovieList`), here's a simplified sequence of what happens:

[Diagram: React Query Flow](#)

1. **Request from Frontend:** Your `HomePage` component calls `useMovieList`, which in turn uses `React Query`'s `useQuery` hook.
2. **Cache Check:** `React Query` first looks into its internal memory (the "cache") for data associated with the `queryKey` (e.g., `["movies", "trending"]`).
3. **Instant Delivery (if fresh):** If the data is found and is considered "fresh" (not `stale`), `React Query` immediately returns it to your component. This is why navigation feels so fast!
4. **Fetching (if stale/missing):** If the data isn't in the cache, or it's `stale` (meaning it's old and needs updating), `React Query` sends a request to your backend API. It also sets `isLoading` to `true`.
5. **Backend & Database:** Your backend API processes the request, retrieves data (e.g., from an external service like TMDb as we'll see in [Chapter 4: Third-Party Movie Data \(TMDb\)](#) or its own database), and sends it back.
6. **Caching & Update:** `React Query` receives the fresh data, stores it in its cache, and then provides it to your component, setting `isLoading` to `false`.
7. **Background Re-fetching:** Even after delivering cached data, `React Query` is smart. It might silently re-fetch data in the background if it determines the data is `stale`, ensuring your app is always showing up-to-date information without interrupting the user.

queryClient Configuration

You can configure how long `React Query` considers data "fresh" or `stale` globally in its `QueryClient`.

```
// apps/frontend/src/lib/query-client.ts
import { QueryClient } from '@tanstack/react-query';

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 5 // Data is considered fresh for 5 minutes
    },
  },
});
```

Here, `staleTime: 1000 * 60 * 5` means that once data is fetched, `React Query` will consider it "fresh" for 5 minutes. If a component requests this data again within 5 minutes, it will get the cached version instantly. After 5 minutes, it will still show the cached version, but it will also trigger a background re-fetch to get the newest data.

useQuery Hook Details

The `useQuery` hook itself specifies a `queryKey` and a `queryFn`.

```
// apps/frontend/src/hooks/useMovies.ts (Inside useMovieList)
import { useQuery } from '@tanstack/react-query';
import { moviesService } from '@service/movies.service';

export function useMovieList() {
  const { data: response, isLoading, isError, error, refetch } = useQuery({
```

```

    queryKey: ["movies", "trending"], // Unique key for this data
    queryFn: () => moviesService.getTrending(), // Function that fetches the data
  });

  const list = response?.results ?? []; // Extract the actual movie list

  return { data: list, isLoading, isError, error, refetch };
}

```

- **queryKey**: This is a unique array (like `["movies", "trending"]`) that **React Query** uses to identify and manage the cached data for this specific query. If any part of the **queryKey** changes (e.g., `["movies", "search", "Spiderman"]` vs `["movies", "search", "Batman"]`), **React Query** treats it as a completely new query.
- **queryFn**: This is the function that actually makes the API call to your backend (e.g., `moviesService.getTrending()`). **React Query** will call this function when it needs to fetch new data.

Zustand (**useAuthStore**): The Simple Global Memory

Zustand is a lightweight library for managing global state. For **useAuthStore**, it acts like a simple container for our user's public details.

```

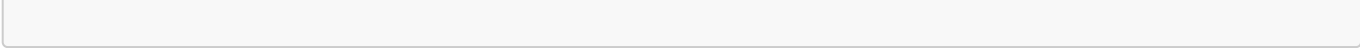
// apps/frontend/src/stores/auth.store.ts (Simplified)
import { create } from "zustand";
import { persist } from "zustand/middleware"; // To save state even if browser
closes

export interface AuthUser {
  userId: number;
  email: string;
}

interface AuthState {
  user: AuthUser | null;
  setUser: (user: AuthUser) => void;
  clearAuth: () => void;
}

export const useAuthStore = create<AuthState>()(
  persist( // This middleware makes the state persist in browser's local storage
    (set) => ({
      user: null, // Initial state: no user logged in
      setUser: (user) => set({ user }), // Action to update the user
      clearAuth: () => set({ user: null }), // Action to clear the user
    }),
    {
      name: "cineconnect-auth", // Key for local storage
      partialize: (state) => ({ user: state.user }), // Only save 'user' to local
storage
    }
  )
);

```



- `create<AuthState>()`: This is the core **Zustand** function that creates our "store." We tell it what shape our state will have (**AuthState**).
- `persist(...)`: This is a **Zustand** "middleware" that automatically saves a part of our store to the browser's **localStorage**. This means if you close your browser and reopen it, **useAuthStore** will remember if you were logged in and who you were (based on the **user** object), providing a smoother experience.
- `user: null`: This is the initial value for our **user** state.
- `setUser` and `clearAuth`: These are functions (called "actions") that allow components to update or reset the **user** state in the store. When these actions are called, any component using **useAuthStore** will automatically re-render with the new **user** data.

This setup provides a simple, efficient way to share the logged-in user's public details across the entire application, avoiding the need for **React Query** for this specific, relatively static piece of global state.

React Query vs. Zustand: A Quick Overview

Here's a comparison of how these two tools serve different purposes in CinéConnect:

Feature	React Query (e.g., <code>useMovieList</code>)	Zustand (<code>useAuthStore</code> via <code>useAuth</code>)
Primary Use	Fetching, caching, and managing asynchronous (remote) data (e.g., movie lists, user profiles from backend).	Managing synchronous (local) global state (e.g., current login status, active user email).
Data Source	Backend API or external services	Frontend application's memory (optionally persisted in localStorage).
Key Benefits	Caching, background re-fetching, loading states, error handling, automatic retries.	Simple global state access, lightweight, easy to update.
Analogy	Data fetching & caching expert	Global recipe book for essential details
Example Data	Trending movies, search results, movie details, user's followers list.	Logged-in user's ID and email, authentication status.

Conclusion

In this chapter, we've demystified how CinéConnect intelligently handles its frontend data and state. We learned that **React Query** acts as our smart data fetching and caching expert, efficiently retrieving movie data, remembering it, keeping it fresh, and gracefully managing loading and error states. Simultaneously, **Zustand**, through **useAuthStore**, provides a lightweight global memory for essential user details, ensuring login status and user information are instantly available across the app. This combination ensures a fast, responsive, and robust user experience.

Now that we understand how to manage our frontend data, in the next chapter, we'll dive deeper into where much of that movie data comes from: **Third-Party Movie Data (TMDB)**.